Lab Experiment Report

Java Programming Laboratory

Basic Information

Title:	ArrayList Operations using the Java Collections Framework

Objective / Aim

To write and execute a Java program that demonstrates the commonly used operations of the **ArrayList** class from the Java Collections Framework, including adding, inserting, retrieving, replacing, searching, removing, sorting, and clearing elements.

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

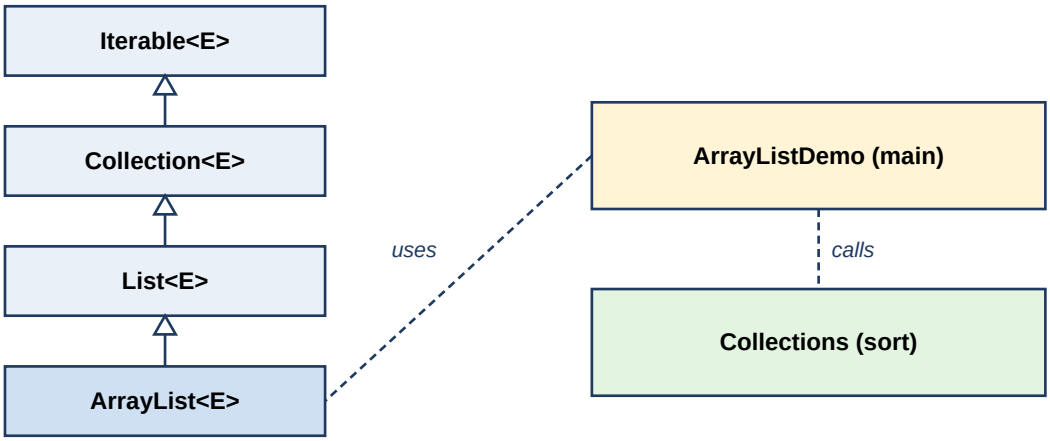
1. Import `java.util.ArrayList` and `java.util.Collections`.
2. Create an `ArrayList<Integer>` object named `list`.
3. Add the elements 10, 20, 30, 20 using `add()` and print the list.
4. Insert 15 at index 1 using `add(index, element)`.
5. Retrieve the element at index 2 using `get()`.
6. Replace the element at index 0 with 5 using `set()`.
7. Check whether the list contains 20 using `contains()`.
8. Display the number of elements and whether the list is empty using `size()` and `isEmpty()`.
9. Find the first and last positions of 20 using `indexOf()` and `lastIndexOf()`.
10. Remove the element at index 0 using `remove(int index)`.
11. Remove the value 30 using `remove(Integer.valueOf(30))`.
12. Sort the list in ascending order using `Collections.sort()`.
13. Remove all elements using `clear()` and display the empty list.

ArrayList Methods Used

Method	Description	Return Type
`add(e)`	Appends an element to the end of the list	boolean

Method	Description	Return Type
`add(index, e)`	Inserts an element at the given index	void
`get(index)`	Returns the element at the given index	E
`set(index, e)`	Replaces the element at the given index	E (old value)
`contains(o)`	Checks whether the element exists	boolean
`size()`	Returns the number of elements	int
`isEmpty()`	Checks whether the list has no elements	boolean
`indexOf(o)`	Index of first occurrence (-1 if absent)	int
`lastIndexOf(o)`	Index of last occurrence (-1 if absent)	int
`remove(index)`	Removes the element at the given index	E
`remove(Object)`	Removes the first matching element	boolean
`Collections.sort(list)`	Sorts the list in ascending order	void
`clear()`	Removes all elements from the list	void

Class Hierarchy Diagram



Relationship: `ArrayList` implements `List`, which extends `Collection` and `Iterable`. `ArrayListDemo` uses an `ArrayList<Integer>` and calls `Collections.sort()`.

Source Code

File: `practise/ArrayListDemo.java`

```
package practise;
import java.util.ArrayList;
import java.util.Collections;

class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();

        // Add elements
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(20);
        System.out.println("ArrayList: " + list);

        // Insert at index
        list.add(1, 15);
        System.out.println("After insert: " + list);

        // Get element
        System.out.println("Element at index 2: " + list.get(2));

        // Replace element
        list.set(0, 5);
        System.out.println("After set: " + list);

        // Check element
        System.out.println("Contains 20: " + list.contains(20));

        // Size and empty check
        System.out.println("Size: " + list.size());
        System.out.println("Is empty: " + list.isEmpty());

        // Find indexes
        System.out.println("First index of 20: " + list.indexOf(20));
        System.out.println("Last index of 20: " + list.lastIndexOf(20));

        // Remove element by index
        list.remove(0);
        System.out.println("After remove index 0: " + list);

        // Remove element by value
        list.remove(Integer.valueOf(30));
        System.out.println("After remove value 30: " + list);

        // Sort
        Collections.sort(list);
        System.out.println("Sorted list: " + list);

        // Clear all elements
        list.clear();
        System.out.println("After clear: " + list);
    }
}
```

Compilation & Execution Commands

Save the source code as `ArrayListDemo.java` inside a folder named `practise` (the file declares `package practise;`). Run the commands from the parent folder of `practise`.

Compile the program:

```
javac practise/ArrayListDemo.java
```

Run the compiled program:

```
java practise.ArrayListDemo
```

Output Screen

```
ArrayList: [10, 20, 30, 20]
After insert: [10, 15, 20, 30, 20]
Element at index 2: 20
After set: [5, 15, 20, 30, 20]
Contains 20: true
Size: 5
Is empty: false
First index of 20: 2
Last index of 20: 4
After remove index 0: [15, 20, 30, 20]
After remove value 30: [15, 20, 20]
Sorted list: [15, 20, 20]
After clear: []
```

Result

The **ArrayList** program was successfully implemented and executed in Java. It demonstrated adding, inserting, accessing, replacing, searching, removing, sorting, and clearing elements, and the output matched the expected behaviour of each operation.